arXiv:2511.17826v1 [cs.LG] 21 Nov 2025

DETERMINISTIC INFERENCE ACROSS TENSOR PARALLEL SIZES THAT ELIMINATES TRAINING–INFERENCE MISMATCH

Ziyang Zhang [*1] **Xinheng Ding** [*2] **Jiayi Yuan** [3] **Rixin Liu** [3] **Huizi Mao** [4] **Jiarong Xing** [3] **Zirui Liu** [2]

ABSTRACT

Deterministic inference is increasingly critical for large language model (LLM) applications such as LLM-as-a-judge evaluation, multi-agent systems, and Reinforcement Learning (RL). However, existing LLM serving frameworks exhibit non-deterministic behavior: identical inputs can yield different outputs when system configurations (e.g., tensor parallel (TP) size, batch size) vary, even under greedy decoding. This arises from the non-associativity of floating-point arithmetic and inconsistent reduction orders across GPUs. While prior work has addressed batch-size–related nondeterminism through batch-invariant kernels, determinism across different TP sizes remains an open problem, particularly in RL settings, where the training engine typically uses Fully Sharded Data Parallel (i.e., TP = 1) while the rollout engine relies on multi-GPU TP to maximize the inference throughput, creating a natural mismatch between the two. This precision mismatch problem may lead to sub-optimal performance or even collapse for RL training. We identify and analyze the root causes of TP-induced inconsistency and propose Tree-Based Invariant Kernels (TBIK), a set of TP-invariant matrix multiplication and reduction primitives that guarantee bit-wise identical results regardless of TP size. Our key insight is to align intra- and inter-GPU reduction orders through a unified hierarchical binary tree structure. We implement these kernels in Triton and integrate them into vLLM and FSDP. Experiments confirm zero probability divergence and bit-wise reproducibility for deterministic inference across different TP sizes. Also, we achieve bit-wise identical results between vLLM and FSDP in RL training pipelines with different parallel strategy. Code is available at https://github.com/nanomaoli/llm_reproducibility.

1 INTRODUCTION

Large language models (LLMs) are increasingly powering various real-world applications (Chang et al., 2024; Yuan et al., 2025b). In many of these scenarios, reproducibility is essential, meaning the generation process must be deterministic: given a fixed random seed and identical inputs, the model should produce the same outputs across different system configurations and different runs.

We emphasize that deterministic inference is particularly crucial in three contexts. *(1) Evaluation with LLM-as-a-judge* (Zheng et al., 2023; Li et al., 2024; Thakur et al., 2025), where LLMs are used as evaluators to assess the quality of other models. If the judge model produces inconsistent evaluation for same inputs, the comparison becomes unreliable. *(2) Multi-agent systems*, where multiple LLMs collaboratively complete one task (Guo et al., 2024). The non-determinism of one model can cascade across the entire system, producing divergent trajectories and making debugging challenging (tse Huang et al., 2025). *(3) Reinforcement Learning (RL) with LLMs* (Sheng et al., 2025), where training and rollout usually use different software stacks (e.g., Fully Sharded Data Parallel (FSDP) (Zhao et al., 2023) vs. vLLM (Kwon et al., 2023)) and configurations (e.g., different tensor-parallel (TP) sizes). As shown in Figure 1, the probabilities computed by the training and rollout engines exhibit a large gap, which can significantly affect the learning process and make an on-policy RL process behave more like off-policy (Yao et al., 2025b).

Unfortunately, prior studies have shown that determinism is a *missing* property in today's LLM systems, under both greedy decoding and random sampling. When changing system configuration, such as batch size and TP size (Yuan et al., 2025a; He & Lab, 2025), the same input will lead to different results. Even with greedy decoding, which should be deterministic since it always selects the most probable next token, reasoning models like DeepSeek-R1-Distill-Qwen-7B can still show up to a 9% variation in accuracy on the AIME dataset (Yuan et al., 2025a). Also, under random sampling, the hardware and system configurations introduce extra variance, making it hard to compare

[*]Equal contribution [1]Independent Researcher [2]University of Minnesota, Minneapolis, Minnesota, USA [3]Rice University, Houston, Texas, USA [4]NVIDIA Corp., Santa Clara, California, USA. Correspondence to: Zirui Liu <zrliu@umn.edu>.

Preprint. Under review.

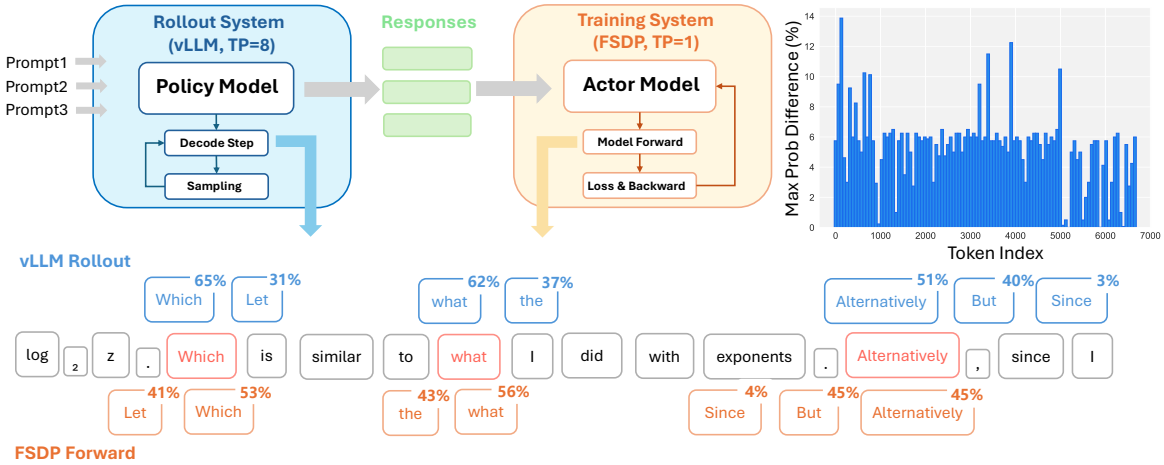


Figure 1. The illustration of numerical variance in LLM-based RL training. Different frameworks and tensor-parallel settings lead to noticeable probability discrepancies for the same model, making it difficult to achieve stable and truly on-policy reinforcement learning.

different methods.

The fundamental reason is that floating point (FP) arithmetic is non-associative, which means processing numbers in different orders can affect the final result due to accumulated rounding errors. However, in serving systems, GPU kernels often consume numbers in varying orders for several reasons including: *(1) continuous batching* (Yu et al., 2022) which dynamically changes the set of requests in a batch; *(2) different implementations of operations*, such as the use of Split-K versus Non-Split-K matrix multiplication (MatMul) (NVIDIA Corporation); *(3) hyperparameters of operations*, like the block size for MatMul kernels and Flash-Attention (Dao et al., 2022); *(4) collective operations in parallel systems*, like All-Reduce; *(5) parallel strategies* like TP, which distribute workloads across multiple GPUs.

To address this issue, (He & Lab, 2025) introduced *batch-invariant operations*, including batch-invariant FlexAttention (Dong et al., 2024), RMSNorm, and MatMul kernels, which guarantees that inference results remain deterministic regardless of batch sizes. It achieves this by ensuring the computation is parallelized along the batch dimension, which disentangles the batch size from the final results of individual requests. However, as its name indicated, this technique is currently limited to only eliminating variances from batch sizes. Yet, as discussed above, many other factors, often even harder to control, can also contribute to inference non-determinism.

In this work, we aim to achieve *fully* deterministic LLM inference. This requires overcoming all sources of non-determinism, with the most critical and challenging arising from the TP and its use of *All-Reduce operations*. In practice, TP configurations often differ across environments due to varying hardware availability and performance requirements. For example, in RL training pipelines, the rollout engine (e.g., vLLM) typically uses multi-GPU TP for fast inference, while the training engine (e.g., FSDP) often runs with TP=1. Such an TP mismatch causes significant result inconsistency, impacting the RL training process.

However, achieving TP-invariant is challenging. Under TP, the MatMul workload is sharded across multiple GPUs. In a typical row-parallel layer in vLLM, each GPU computes a *partial matrix multiplication* result for its local shard of the model weights, and these partial results must then be summed together through an *All-Reduce operation*. As shown in Figure 7, the issue is that the accumulation order of floating-point numbers are affected by both the TP size and the communication pattern used (e.g., NCCL's ring or tree reduction). Thus, when the TP configuration changes, the effective arithmetic sequence of additions also changes, producing different outputs. To achieve TP-invariant results, we must align the accumulation order between the local matrix multiplication and the global All-Reduce so that the whole inference follows a consistent reduction sequence, regardless of how many GPUs are used.

As shown in Figure 7, our key idea is to leverage a hierarchical tree-structured reduction that aligns the accumulation order across GPUs. By enforcing the same binary-tree topology for both intra- and inter-GPU reductions, the computation order remains consistent under different TP sizes. Based on this idea, we design and implement a tree-based MatMul kernel that aligned with the tree all-reduce. Specifically, the kernel decomposes each matrix multiplication into tiles along the reduction dimension and applies a fixed-depth binary reduction tree over these tiles as shown in Figure 6. Each GPU first performs a deterministic local reduction following the same tree structure, producing partial results that are then aggregated across devices using the identical hierarchical pattern. This unified tree topology ensures that,

regardless of the TP size or GPU count, every reduction operation follows an identical arithmetic sequence.

In addition, to align vLLM and FSDP in RL pipeline with different TP sizes, we eliminate other sources of non-determinism between vLLM and FSDP by enforcing the use of consistent operation implementations (e.g., prefill-only TritonAttention in both systems and the same batch-invariant kernels) and identical operation hyperparameters (e.g., block sizes in MatMul and TritonAttention). Combined with our TP-Invariant kernels, these measures ensure full determinism and alignment across both systems, even when using different configurations on the same type of GPU. In summary, we:

- Identify the problem of non-determinism under different TP sizes and analyze its impact on benchmark evaluation and RL training.
- Propose **T**ree **B**ased **I**nvariant **K**ernels (**TBIK**) that aligns MatMul with the inter-GPU All-Reduce structure. It eliminates the numerical variance caused by different TP sizes and All-Reduce orders.
- Achieve fully deterministic inference by implementing the proposed kernels in Triton and integrating them with both vLLM and FSDP, while ensuring consistency in other factors such as operation implementations and hyperparameters. Our experiments demonstrate bit-wise identical results across the two systems, providing a foundation for truly on-policy RL training.

2 Background

2.1 IEEE 754 FP Operations are Non-Associative

A widely known issue associated with IEEE 754 floating point operations is that they are *non-associative*, namely, $(a+b)+c \neq a+(b+c)$ (Yuan et al., 2025a). This means that the order in which numbers are added can affect the final result due to accumulated rounding errors.

Many algorithm and system-level efforts are employed to mitigate this issue. Fused Multiply-Add (FMA) (NVIDIA Corporation, 2025) is used to compute $x*y+z$ with an exact double-length product, followed by an addition with a single rounding. In Matrix Multiplication (MatMul) and Flash-Attention (Dao et al., 2022) kernels, a common practice is to use FP32 accumulator for accumulating intermediate results, even when the input data is in a lower precision format. Other commonly used operations like Softmax, LayerNorm, RMSNorm, and RoPE, are all set to FP32 precision by default.

2.2 The Sources of Non-Determinism

In modern serving systems, there are several factors that can change the order of FP operations, thus affecting final results. *(1) Continuous batching* (Yu et al., 2022), which dynamically changes the set of requests in a batch and the batch size. *(2) Different implementations of operations*, such as the use of Split-K versus Non-Split-K Matmul (NVIDIA Corporation), yield non-deterministic results because Split-K requires accumulating partial sums whose combination order varies based on thread scheduling. *(3) Hyperparameters of operations*, like the block size for MatMul and Flash-attention, also change the specific sequence of accumulation steps within a kernel. *(4) Collective operations in parallel systems*, like All-Reduce, are often non-deterministic, causing the final aggregated value to differ. *(5) Parallel strategies* like TP, which shard workloads across GPUs. *(6) Different GPU architectures*, which may rely on different low-level instruction sets for matrix multiplication, such as `wgmma` on Hopper.

Due to these factors, prior work (Yuan et al., 2025a) shows that even under greedy decoding, inference outputs can diverge significantly across different batch sizes, GPU architectures, and TP configurations.

3 Motivation and Challenges

3.1 Motivation: Non-Deterministism Impact to RL

Non-determinism can impact many scenarios as mentioned in Section 1. Here, we use the most popular area, LLM-based Reinforcement Learning (RL) as an example to showcase its impact. For the ease of illustration, following (Yao et al., 2025a), we use the REINFORCE algorithm as an example which updates the policy π_θ via

$$\theta \leftarrow \theta + \mu \cdot \mathbb{E}_{\underbrace{a \sim \pi(\theta)}_{\text{rollout}}} \left[R(a) \cdot \underbrace{\nabla_\theta \log \pi(a,\theta)}_{\text{training}} \right],$$

where a is the token generated by the policy network π_θ and $R(a)$ is the reward. In practice, rollout generation is executed by inference engines such as vLLM and SGLang (Zheng et al., 2024), while model training uses a separate backend (e.g., FSDP). Such a hybrid design update the parameters as follows:

$$\theta \leftarrow \theta + \mu \cdot \mathbb{E}_{a \sim \pi_{\text{rollout}}(\theta)} \left[R(a) \cdot \nabla_\theta \log \pi_{\text{learner}}(a,\theta) \right].$$

Even if π_{sampler} and π_{learner} share an identical parameters θ, differences in kernel implementations and framework settings (e.g., TP size) can result in large discrepancies in probability calculations as shown in Figure 1, which implicitly converts an on-policy update into an off-policy one (Yao et al., 2025a). This mismatch can destabilize training and negatively affect convergence and policy performance. Note that this problem is much more severe in Mixture-of-Experts

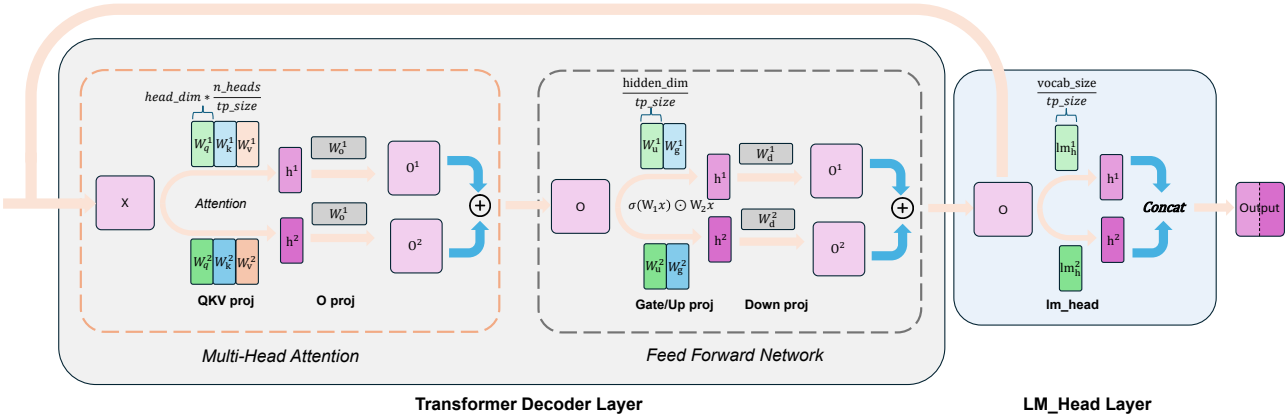


Figure 2. Illustration of tensor-parallel weight partitioning in the Transformer model architecture (e.g., Qwen3 Dense) under the vLLM tensor-parallel setting. In this configuration, the *QKV proj*, *gate proj*, *up proj*, and *lm head* layers are *column-parallel*, while the *o proj* and *down proj* layers are *row-parallel*. For non-matmul operations such as RMSNorm and RoPE, parameters and computations are not split across GPUs, and these layers are omitted from the figure for clarity.

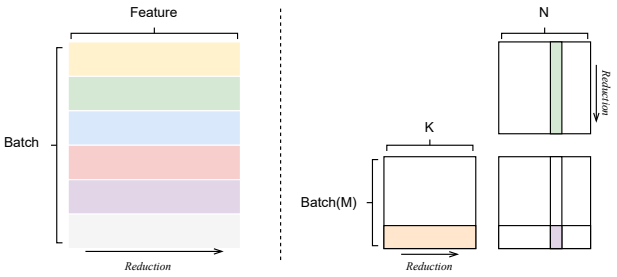


Figure 3. Implementations in Batch Invariant Operations. Left: RMSNorm. Right: MatMul.

(MoE) models (Fedus et al., 2022; Yang et al., 2025) compared to dense models, as a small perturbation in probability may cause the router to select entirely different experts.

3.2 Existing Work: Batch-Invariant Operations

To address this problem, (He & Lab, 2025) introduced *batch-invariant operations* (BIO), including batch-invariant FlexAttention, RMSNorm, and MatMul kernels, which guarantees that inference results remain deterministic regardless of batch sizes. Specifically, it achieves this by parallelizing the computation along the batch dimension and making each batch element compute independently, which guarantees a fixed reduction order regardless of the batch size. For example, in RMSNorm, each batch element is processed on a single compute core, eliminating inter-core communication for feature-dimension reductions. In MatMul, each core computes the dot products for a 2D tile and performs the full reduction locally, following a non-split-K strategy. See Figure 3 for a schematic illustration.

3.3 Batch-Invariant Operations Meet Parallelisms

However, "batch-invariant" is only the first step towards achieving fully deterministic inference. As we explained in Section 2.2, there are many other factors contributing to non-determinism. The most important ones are those related to parallelisms. To understand the problem, here, we first provide a brief overview of the three most commonly used parallel strategies in serving systems and discuss how batch-invariant operations behave under each of them.

Data Parallelism (DP). Each GPU holds a complete copy of the model but processes a different subset of batch samples, which effectively shards the workload along the batch-size dimension. Changing the degree of DP parallelism is therefore equivalent to changing the batch size, and batch-invariant operations ensure results to remain consistent.

Pipeline Parallelism (PP). Different layers of the model are distributed across multiple GPUs or nodes, **which is the default inter-node parallelization strategy**. The intermediate activations are sequentially passed to the next stage to complete the forward computation. Changing the PP degree does not alter the computation order within each sample; therefore, as long as deterministic communication operations are used[1], the model's outputs remain deterministic.

Tensor Parallelism (TP). The weight matrix of each layer is sharded across multiple GPUs, and the complete output of the layer is obtained by aggregating the partial results computed on different GPUs. **TP is the default intra-node parallelization strategy**. As we analyze later in Section 3.4, changing the TP configuration can alter the model's outputs,

[1]In NCCL, users can specify node topology https://docs.nvidia.com/deeplearning/nccl/user-guide/docs/env.html

and batch-invariant kernels are not effective in handling such variations. We run the same prompt from AIME24 dataset across varying TP sizes (1/2/4/8) and check if output token sequence remain identical. Table 1 shows the average count of unique outputs, a value of 4 indicates that the outputs for each prompt in the dataset are different across all TP configurations. Moreover, divergences in the Qwen3-8B model outputs caused solely by changes in TP size can lead to more than 4% variance in accuracy on the AIME24 dataset, as shown in Figure 4.

It worth noting that using different TP sizes across environments is very common in practice because **performance optimization are coupled with TP Sizes**. The optimal TP size is usually adjusted based on the required inference speed (throughput/latency) and available hardware resources. The TP value itself may vary in production to meet shifting performance demands.

This variability is especially evident in RL and large-scale training pipelines, **where the training and rollout systems typically employ different frameworks and parallel strategy for performance optimization.** As illustrated in Figure 1, training often uses FSDP with a TP size 1, while rollout engine commonly relies on frameworks such as vLLM with larger TP sizes to maximize inference throughput. This framework-level heterogeneity naturally results in mismatched TP configurations across environments.

Table 1. Average count of unique outputs across four TP sizes (1/2/4/8) under different batch size settings. **Batch-Invariant Operations(BIO) does not guarantee identical outputs when change the TP size.** Model: Qwen3-8B, Task: AIME24.

	BS=8	BS=16	BS=32
BF16	4.00	4.00	4.00
BIO	4.00	4.00	4.00

3.4 Why BIO Fail Under Different TP Sizes?

In this section, we first explain the weight-sharding strategy employed in tensor parallelism and then identify the underlying reasons why batch-invariant operations fail to ensure TP invariance.

Figure 2 illustrates how the weights of transformer models are split under tensor parallel in vLLM. In general, TP distributes workloads in two complementary ways: **Column Parallel** and **Row Parallel**. Specifically, the *QKV proj* in self-attention and the *up proj* and *gate proj* in the feed-forward network (FFN) are implemented as **column-parallel** operations. In column-parallel mode (Figure 5a), the weight matrix $W \in \mathbb{R}^{K\times N}$ is split along its output dimension into C blocks:

$$W_i \in \mathbb{R}^{K\times \frac{N}{C}}, \quad i = 1, \dots, C,$$

where C is the number of GPUs.

Each GPU computes a partial result

$$O_i = XW_i, \quad O_i \in \mathbb{R}^{M\times \frac{N}{C}},$$

and the final output is obtained by concatenating all partial results:

列并行

$$O = XW = \left[\, XW_1 \;\; XW_2 \;\; \cdots \;\; XW_C \,\right].$$

Since these layers expand the feature dimension, their outputs can remain locally partitioned without requiring immediate synchronization.

In contrast, the *o proj* in attention and the *down proj* in the FFN are **row-parallel**. In row-parallel mode (Figure 5b), the input matrix X is split along its columns while the weight matrix W is split along its rows, so that each GPU holds a slice

$$X_i \in \mathbb{R}^{M\times \frac{K}{C}}, \quad W_i \in \mathbb{R}^{\frac{K}{C}\times N}, \quad i = 1, \dots, C,$$

Each GPU computes the partial product X_iW_i, and the final output matrix is obtained by summing all partial results across GPUs:

$$XW = \sum_{i=1}^{C} X_iW_i.$$

行并行

This design requires a collective `all-reduce` operation to merge the partial results, introducing communication overhead and making the output sensitive to the order of floating-point summation due to non-associativity.

The rationale behind this design is rooted in how data flows through Transformer layers. The outputs of column-parallel layers (e.g., the *QKV proj*, *up proj*, and *gate proj*) serve as inputs to subsequent row-parallel layers (e.g., the *o proj* and *down proj*). Because the column-parallel layers naturally partition their outputs along the feature dimension, these partitions can be directly consumed by the row-parallel layers without additional synchronization. However, the row-parallel linear layer, which serves as the final layer in both the self-attention and FFN modules, must aggregate results from all GPUs through `all-reduce` operations to obtain the complete output of the current module.

Changing the TP size alters the computation order of the row-parallel layer. For the column-parallel layer, changing the TP size does not affect the output results, since the layer is partitioned along the N dimension, and each GPU produces an output of size $M \times \frac{N}{C}$. These outputs are mutually independent, as no accumulation occurs along the shared dimension. However, in the case of a row-parallel layer, both the input X and the weights W are partitioned across GPUs along the K dimension. Each GPU computes a

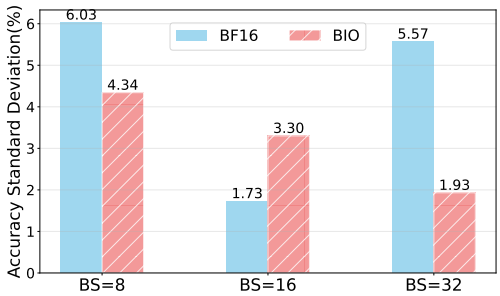


Figure 4. Accuracy standard deviation of Qwen3-8B on AIME24 dataset under different tensor parallelism (TP = 1/2/4/8) settings. While BIO enhances output determinism relative to vanilla BF16 inference, the accuracy variance can still reach over 4%.

partial result of size $M \times N$, and the row-parallel layers aggregate all partial results across GPUs through element-wise reductions. Even if the reduction algorithm itself is deterministic, the number of participating devices can alter the computation order for each element. As illustrated in Figure 7, when using the standard cuBLAS GEMM for matrix multiplication, each GPU first performs a local reduction along the K dimension sequentially, after which the partial results are further reduced across GPUs via NCCL along the same dimension. Thus, different TP sizes change the reduction order, which can lead to divergent outputs. Consequently, batch-invariant kernels alone cannot guarantee reproducibility across different TP configurations.

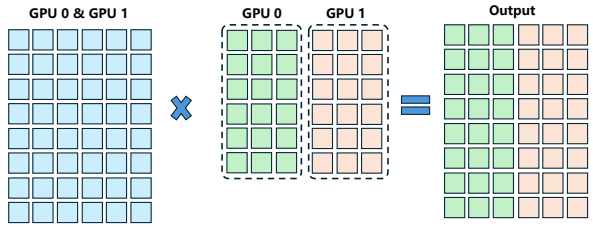
In summary, to achieve deterministic inference regardless of TP size, it is necessary to co-design the matrix multiplication and inter-GPU All-Reduce operations to ensure that the global reduction order of the row-parallel linear layer remains consistent across varying TP configurations.

4 Design and Implementation

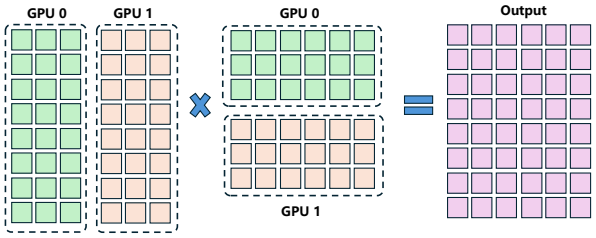
4.1 Kernel Design

To achieve TP-invariant inference, we design a row-parallel linear layer whose reduction order is consistent with that of the All-Reduce operation. Modern matrix multiplication typically divides input matrices into tiles and accumulates partial results along the K-dimension, as illustrated in Figure 6. Each output tile of size $[Block_M, Block_N]$ is obtained by summing over $T_k = \frac{K}{Block_K}$ partial tiles.

Our key idea is to introduce a **hierarchical tree-based reduction structure** that mirrors the accumulation order of Tree All-Reduce. By enforcing an identical binary-tree topology for both intra- and inter-GPU reductions, the computation order remains consistent across different TP sizes. In this structure, each tile corresponds to a leaf node, and internal nodes represent the accumulation sequence. The reduction tree itself is fixed and independent of the number of GPUs, ensuring identical accumulation paths across TP



(a) Column-parallel matrix multiplication.



(b) Row-parallel matrix multiplication.

Figure 5. illustrations of column-parallel and row-parallel Matmul.

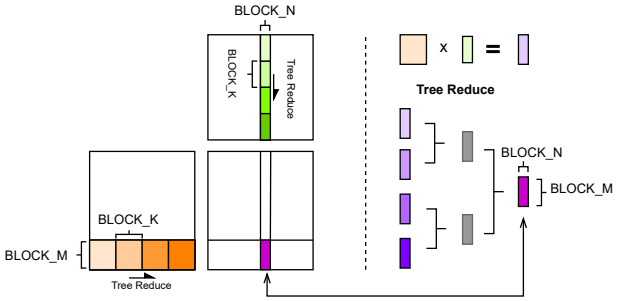


Figure 6. The illustration of our Tree-Reduce MatMul Kernels.

锁死加法的顺序

configurations.

Following this principle, we propose **Tree-Based Invariant Kernels** (**TBIK**) that applies a consistent full binary-tree topology to all tiles. Each GPU is mapped to a specific level of the tree, and both intra- and inter-GPU reductions follow the same hierarchical pattern. This design guarantees deterministic accumulation and consistent numerical outputs regardless of TP configuration.

4.2 Implementation

As shown in Figure 7, our **TBIK** is composed of two stages: *intra-GPU reduction* with Tree-based MatMul kernels and *inter-GPU reduction* with Tree All-Reduce.

Intra-GPU Reduction within Tree-Reduce MatMul Kernels. We implement the intra-GPU reduction using `Triton` (Tillet et al., 2019). As shown in Algorithm 1, each GPU is responsible for $\frac{T_k}{C}$ tiles, where C denotes the total

固定拆分；
TP=6:
GPU0: t0
GPU1: t1
GPU2: t2
GPU3: t3
GPU4: t4, t6
GPU5: t5, t7

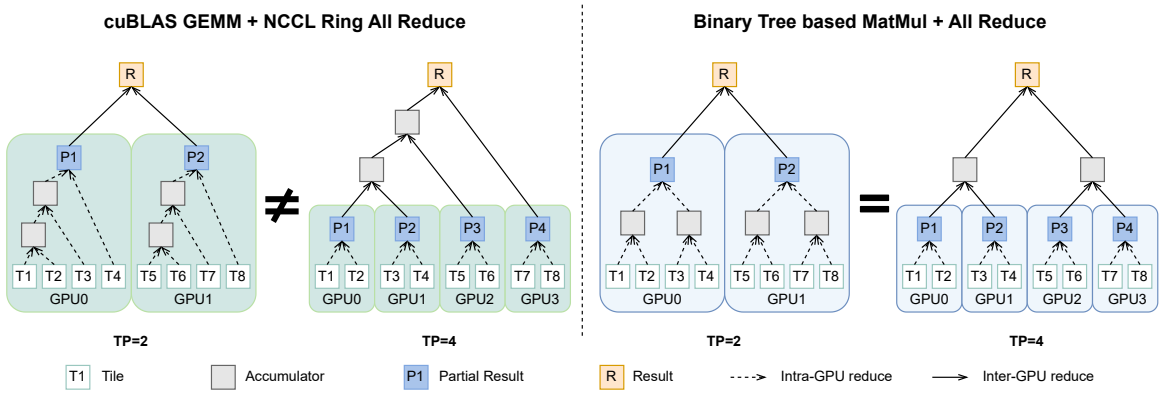


Figure 7. Comparison between the cuBLAS GEMM with NCCL Ring Reduce (left) and our Tree Reduce Kernel (right). In the Ring Reduce, inter-GPU communication follows a ring pattern, while intra-GPU reduction is performed sequentially. In contrast, our Tree Based Invariant Kernel adopts a hierarchical tree structure for both intra- and inter-GPU reductions, ensuring a consistent reduction order across different TP sizes.

number of GPUs. To realize a tree-structured accumulation, intermediate partial results must be temporarily stored. We can show that at least L accumulators are required, where $L = \log_2 \frac{T_k}{C}$ represents the depth of the binary reduction tree. Accordingly, we allocate an accumulator buffer S of shape $[L, Block_M, Block_N]$ for computing each output tile.

During computation, each tile $[Block_M, Block_K]$ from A and $[Block_K, Block_N]$ from B is sequentially loaded and multiplied, and the partial product is accumulated into the first level of S. We maintain a counter tensor $Count$ of shape $[L]$ to record how many tiles have been merged at each level. Whenever a level-l counter reaches two, a *carry-over* operation is triggered: the corresponding two partial sums at level l are reduced into the next level by

$$S[l+1] = S[l+1] + S[l],$$

after which both the value and the counter at level l are cleared. This process repeats until all tiles along the K-dimension are processed, and the final reduced output stores in $S[L]$.

For cases where T_K is not a power of two (e.g., when $K = 6144$ in Qwen3-1.7B `down_proj` and $Block_K$ is a power of two), we introduce an adaptive variable K_{first} that specifies how many tiles are accumulated before the first carry-over. Although this breaks the strict binary reduction pattern at the first level, the overall invariance property still holds as long as $\frac{T_K}{C \times K_{\text{first}}} \geq TP_{\max}$.

Inter-GPU Reduction with Tree All-Reduce. After local accumulation within the MatMul Kernel, we synchronize the per-GPU partial results using an `all-gather` collective operation. The gathered results are then reduced via pairwise summation, ensuring that the inter-GPU reduction also follows the fixed tree structure. More details about our customized all reduce operation can be found in Appendix A.2

4.3 Theoretical Proof

Operator definition. Define $T(\cdot)$ on sequences by recursion on the full binary-tree topology:

$$T(k_1, \ldots, k_{2^t}) = \begin{cases} k_1 \oplus k_2, & t = 1, \\ T\big(T(k_1, \ldots, k_{2^{t-1}}),\ T(k_{2^{t-1}+1}, \ldots, k_{2^t})\big), & t > 1. \end{cases}$$

That is, $T(\cdot)$ performs pairwise reductions following the parent–child structure of $\mathcal{T}^*$, where each internal node applies the operator $\oplus$ to the outputs of its two child subtrees.

Theorem 1. *Let N be the total number of tiles and C be the TP size, where C is a power of two. If the N tiles are evenly partitioned across the C GPUs, then under the operator $T(\cdot)$ defined above, the hierarchical reduction order is fixed regardless of TP sizes.*

Proof sketch. Define $M = N/C$ as the number of tiles per GPU, and assign each GPU $d = 1, \ldots, C$ the contiguous tiles

$$\mathcal{L}_d = \{k_{(d-1)M+1}, \ldots, k_{dM}\}.$$

We prove that

$$T(k_1, \ldots, k_N) = T\big(T(\mathcal{L}_1), \ldots, T(\mathcal{L}_C)\big),$$

where $T(\cdot)$ is the binary-tree operator defined above.

Base case ($C = 1$). When there is only one GPU, $\mathcal{L}_1$ contains all tiles, so

$$T(k_1, \ldots, k_N) = T(\mathcal{L}_1),$$

and the statement trivially holds.

Inductive step. Assume the equality holds for $C/2$ GPUs. For C GPUs, split the GPUs into the first half (blocks $\mathcal{L}_1, \ldots, \mathcal{L}_{C/2}$) and the second half (blocks

$\mathcal{L}_{C/2+1}, \ldots, \mathcal{L}_C$). By the induction hypothesis, the reductions within each half satisfy

$$T(k_1, \ldots, k_{N/2}) = T(T(\mathcal{L}_1), \ldots, T(\mathcal{L}_{C/2}))$$
$$T(k_{N/2+1}, \ldots, k_N) = T(T(\mathcal{L}_{C/2+1}), \ldots, T(\mathcal{L}_C)).$$

The global reduction applies T to these two halves:

$$T(k_1, \ldots, k_N) = T\big(T(k_1, \ldots, k_{N/2}), T(k_{N/2+1}, \ldots, k_N)\big),$$

which by the induction hypothesis equals

$$T\big(T(\mathcal{L}_1), \ldots, T(\mathcal{L}_C)\big).$$

Hence, by recursion, the hierarchical reduction executes the exact same sequence of pairwise $\oplus$ operations as the global reduction, so the result is bitwise-identical. □

5 EVALUATION

We designed our experiments to mainly answer three research questions: **RQ1.** Whether our proposed methods can achieve bit-wise identical results across different TP sizes? **RQ2.** What is the overhead introduced by our methods? **RQ3.** Can our methods fundamentally solve the precision mismatch problem in RL training?

5.1 Evaluation Metrics

We evaluate the reproducibility of model outputs using two metrics: **Count of Unique Outputs**, and **Maximum Probability Divergence**.

Count of Unique Outputs: We count the number of distinct generated token sequences for the same prompt across K runs:

$$U = |\text{unique}(\{y_1, y_2, \ldots, y_K\})|.$$

A value of $U = 1$ indicates outputs are the same across settings.

Maximum Probability Divergence: To verify that our method achieves bit-wise reproducibility, we calculate Δ_i, the maximum divergence of the top-5 predictive probabilities at each position $i \in \{1, \ldots, L\}$ of outputs under K different experimental settings, and the overall divergence is obtained by averaging across all positions. A smaller D indicates stronger agreement among different settings at the probability-distribution level, while bit-wise identical outputs ideally correspond to a value of 0.

5.2 Experiment Setup

We conduct experiments on four models from different families: `Qwen3-8B`, `Qwen3-32B` (Team, 2025), `Mistral-7B-Instruct-v0.3` (Mistral AI, 2024), and `Llama-3.1-8B-Instruct` (Meta AI, 2024). The Qwen models are evaluated under the thinking mode with a maximum output length of 8192, while the instruct models are evaluated with a maximum output length of 2048. We evaluate our models on two commonly used benchmarks, AIME24 (Jia, 2024) and AMC23 (AI-MO, 2024), which test numerical reasoning and mathematical problem-solving capabilities. We evaluate the reproducibility of LLMs under random sampling with fixed random seeds and decoding parameters, which better reflects real-world usage scenarios.

Our experiments evaluate three methods: (1) vanilla BF16 inference, (2) inference using only **B**atch-**I**nvariant **O**perations **(BIO)**, and (3) inference combining our **T**ree **B**ased **I**nvariant **K**ernels with **B**atch-**I**nvariant **O**perations **(BIO+TBIK)**. For **BIO**, we use the implementation from (He & Lab, 2025). For each model–dataset pair, we evaluate under 12 different runtime configurations, representing all combinations of 4 TP sizes (1/2/4/8), and 3 batch sizes (8/16/32), i.e., $4 \times 3 = 12$ configurations, to simulate diverse deployment environments commonly encountered in real-world inference. For the Qwen3-32B model, due to GPU memory limitations, we adopt 9 runtime configurations with 3 TP settings (2/4/8), and 3 BS settings (8/16/32).

We repeat all the above experiments on two different GPU types, i.e., NVIDIA RTX PRO 6000 and NVIDIA L40S, to verify the consistency of results across heterogeneous hardware environments. All experiments are conducted using `vLLM` (Kwon et al., 2023) as the inference backend. More details about experiment setup, please refer to Appendix B.

5.3 Reproducibility Evaluation

As shown in Table 2, under vanilla BF16 inference, the average Count of Unique Outputs is close to the number of different runtime configurations, indicating that changing any single system configuration leads to different outputs. When only BIO are applied, the average Count of Unique Outputs decreases, since consistent outputs are obtained across different batch sizes when TP equals 1 or 2, reflecting the "batch invariance" property[2]. However, this approach fails to maintain consistency when $TP > 2$ or when different TP sizes are used. In contrast, our BIO+TBIK configuration consistently yields an average Count of Unique Outputs of 1, demonstrating that the model produces identical inference outputs across all system configurations.

Table 3 presents the average Maximum Probability Divergence on two datasets. Compared with vanilla BF16 inference, BIO slightly reduces the fluctuation of token pre-

[2]To clarify, BIO ensures consistent results across different batch sizes when running with either TP=1 or TP=2. However, the outputs produced under TP=1 and TP=2 are not identical to each other.

Table 2. **Average Count of Unique Outputs on AIME24 and AMC23.** For each prompt, outputs are generated under 12 runtime configurations (BS=8/16/32; TP=1/2/4/8). Qwen3-32B was evaluated under 9 configurations (BS=8/16/32; TP=2/4/8). "1" indicates identical outputs across all configurations.

Model	Method	AIME'24	AMC'23
Qwen3-8B	BF16	12.00	10.85
	BIO	7.87	7.78
	BIO+TBIK	**1**	**1**
Mistral-7B-Instruct	BF16	12.00	11.08
	BIO	7.97	7.75
	BIO+TBIK	**1**	**1**
Llama-3.1-8B-Instruct	BF16	9.60	9.85
	BIO	7.20	7.13
	BIO+TBIK	**1**	**1**
Qwen3-32B	BF16	9.00	8.00
	BIO	6.90	6.75
	BIO+TBIK	**1**	**1**

Table 3. **Average Maximum Probability Divergence on AIME24 and AMC23($\times 10^{-3}$).** A larger value indicates greater numerical discrepancies, while 0 means the predicted token probability distributions are bitwise identical across all evaluated runtime configurations.

Model	Method	AIME'24	AMC'23
Qwen3-8B	BF16	10.01	6.79
	BIO	9.10	6.90
	BIO+TBIK	**0**	**0**
Mistral-7B-Instruct	BF16	17.37	19.88
	BIO	14.10	15.87
	BIO+TBIK	**0**	**0**
Llama-3.1-8B-Instruct	BF16	26.48	31.09
	BIO	27.54	21.06
	BIO+TBIK	**0**	**0**
Qwen3-32B	BF16	8.01	9.80
	BIO	7.79	8.97
	BIO+TBIK	**0**	**0**

dictive probabilities due to its parallel strategy and control over reduction operations. However, it still fails to handle variations caused by changes in TP. In contrast, BIO+TBIK achieves a strictly zero Maximum Probability Divergence across all experimental settings, demonstrating bit-wise deterministic LLM inference. The above results were obtained on NVIDIA L40s GPUs, more experimental results on NVIDIA RTX PRO 6000 GPUs can be found in the Appendix C.

5.4 Performance Evaluation

In this section, we present the performance evaluation of our TP-Invariant Kernel, as well as the end-to-end latency after integrating TBIK into the vLLM backend.

Kernel-Level Comparison. For the kernel throughput analysis shown in Figure 8, we observe that our TP-Invariant kernel runs slower than cuBLAS at small M due to the fixed block-size constraint, which introduces noticeable computation overhead. As the batch size increases, our kernel achieves 63% of cuBLAS performance, reaching around 120 TFLOPS in BF16 compared to 190 TFLOPS for cuBLAS. This overhead primarily stems from the additional temporary accumulators required for tree-reduction operations, which increase both I/O and arithmetic operations. Under the FP32 setting, the impact of these overheads is mitigated, and both kernels achieve comparable performance. **We emphasize that our current implementation of tree-based MatMul kernels primarily serves to demonstrate that TP-Invariant deterministic inference is achievable.** The performance can be further improved by incorporating advanced optimizations such as block size tuning, and warp specialization, which have been shown to significantly enhance performance (Yu et al., 2025).

End-to-End Latency Comparison. We measured the end-to-end latency of the Qwen3-8B model using four NVIDIA L40s GPUs (TP=4) across different input and output lengths, with a batch size of 64. We use the official bench- mark scripts provided in the vLLM repository. For a fair comparison, the vanilla BF16 baseline uses TritonAtten- tion as the backend, with CUDA Graph and Prefix Caching disabled. As shown in Figure 9, fully deterministic inference using BIO+TBIK introduces substantial overhead compared with vanilla BF16 inference. The overhead can be clearly attributed to two components: the BIO module alone contributes about $18 \sim 99\%$ overhead relative to BF16, while pure TBIK (the additional cost on top of BIO) contributes another $24 \sim 38\%$ relative to BF16. Taken together, these effects result in a total BIO+TBIK overhead ranging from 56% to 135% compared to the vanilla BF16 baseline. For TBIK, the contributors to this cost are the lower throughput of the unoptimized tree-based MatMul compared to cuBLAS BF16, and the sub-optimal deterministic tree-based all-reduce operation we employ. A fine-grained decomposition of TBIK's computational costs can be found in Appendix D. **It reveals that tree-based MatMul kernel accounts for only $3 \sim 14\%$ of the overhead. In comparison, the All-Reduce operation incurs $28 \sim 50\%$ overhead, mainly due to our naive implementation and the absence of NVLink in the test environment, which significantly reduces communication efficiency.**

In summary, we show that performance of TP-Invariant deterministic inference is acceptable. Despite the substantial overhead introduced by deterministic inference compared with normal-mode execution, this trade-off is indispensable

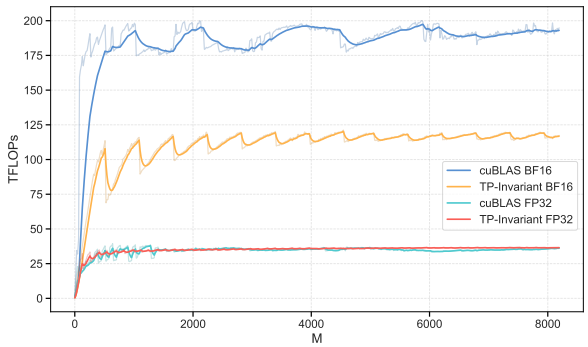


Figure 8. Throughput comparison of TP-Invariant and cuBLAS Matmul kernels for varying M, with $K = 6144$ and $N = 2048$.

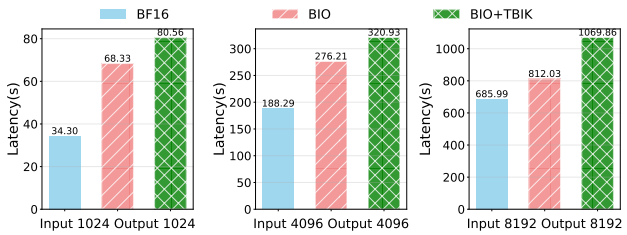


Figure 9. End-to-end latency of the Qwen3-8B model on four NVIDIA L40s GPUs under varying input/output lengths with a batch size of 64.

for scenarios that demand reliable debugging and reproducible results. The performance can be further enhanced by applying advanced optimizations to the tree-based MatMul kernel and All-Reduce components of TBIK, in addition to improved integration and more comprehensive optimization and support for the BIO module within vLLM.

5.5 Bridging the Probability Gap Between vLLM and FSDP in RL

Here we integrate TBIK into the RL training pipeline to solve the probability mismatch problem between training and rollout engines. Specifically, we patch our kernels to both vLLM and FSDP. Below we introduce the implementation details besides our kernels to make sure they are aligned. To eliminate these discrepancies, we introduce the following framework-level modifications for both vLLM and FSDP:

1. **Linear Layers.** We replace standard linear layers with customized kernels to ensure both batch and TP invariance. Specifically, we employ BIO for column-parallelized layers in vLLM (`qkv_proj`, `up_proj`, `gate_proj`, and `lm_head`), and our TP-Invariant kernel for row-parallelized layers (`down_proj` and `o_proj`).

2. **Attention.** To ensure determinism, we fix the tile size in TritonAttention to enable batch invariance. Additionally, we disable chunked prefill in vLLM and enforce both the prefill and decode phases to use the same attention kernel. This adjustment is necessary because FSDP training only involves the prefill phase; therefore, we disable these decoding-side optimizations in vLLM. The same modified attention backend is also applied to FSDP.

3. **Other Kernels.** For other kernels, including RMSNorm, RoPE embedding, and SiLU activation, we used the same kernel implementations as in vLLM for FSDP to ensure consistency.

To this end, we achieve full determinism across both **vLLM** and **FSDP**. To measure per-token probability differences, we select 4 sample prompts from AIME and generate responses using vLLM with a maximum output length of 8192 tokens. The generated responses are then treated as a batch and fed once into FSDP to compute token probabilities. We evaluate the maximum per-token probability gap between the vLLM-generated responses and the corresponding FSDP outputs. The distribution of these maximum per-token gaps is shown in Figure 10.

From the results, we observe that BIO partially reduces the probability gap, though a noticeable discrepancy still remains. In contrast, after applying our TBIK method, the probabilities predicted by vLLM and FSDP become bit-wise identical, fully closing the gap between the two frameworks and effectively eliminating the precision mismatch problem.

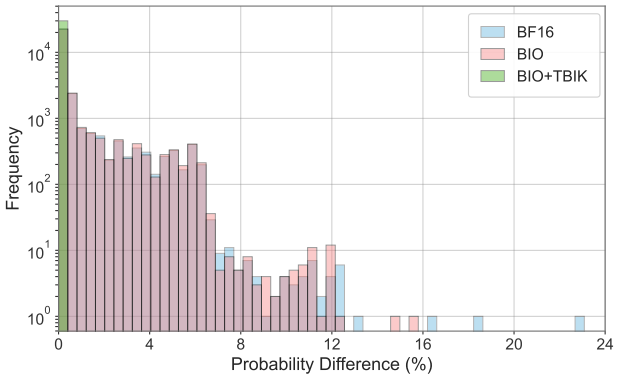


Figure 10. Maximum per-token probability difference calculated by vLLM (TP=4) and FSDP (TP=1) on Qwen3-8B using four NVIDIA L40S GPUs. Our method achieves zero probability gaps.

6 Conclusion and Future Work

We presented TBIK, a framework for achieving deterministic LLM inference across different TP sizes. By aligning intra- and inter-GPU reduction orders through a unified binary tree topology, TBIK eliminates nondeterminism caused

by floating-point non-associativity and inconsistent reduction patterns. Integrated into Triton, vLLM, and FSDP, our approach delivers bit-wise identical outputs across batch sizes, TP configurations, and frameworks. Experiments confirm zero probability divergence and reproducibility across heterogeneous hardware, bridging the long-standing precision gap between vLLM and FSDP in RL pipelines.

In the future, besides further optimizing the kernel performance, an important next step is extending this guarantee to low-bit matmul kernels used in modern quantized LLMs (Lin et al., 2024; Frantar et al., 2022; Liu et al., 2024; Yuan et al., 2024). Quantization introduces additional nondeterministic factors, such as rounding, scaling, and fused dequantization paths, that must be aligned with the same tree-structured accumulation used in TBIK. Designing a tree-consistent low-bit GEMM that preserves determinism while retaining the efficiency benefits of low bit inference remains a key direction for future work.

References

AI-MO. Aimo validation amc. Hugging Face Datasets, 2024. URL https://huggingface.co/datasets/AI-MO/aimo-validation-amc. Dataset; 83 rows; Apache-2.0 license.

Chang, Y., Wang, X., Wang, J., Wu, Y., Yang, L., Zhu, K., Chen, H., Yi, X., Wang, C., Wang, Y., et al. A survey on evaluation of large language models. *ACM transactions on intelligent systems and technology*, 15(3):1–45, 2024.

Dao, T., Fu, D., Ermon, S., Rudra, A., and Ré, C. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in neural information processing systems*, 35:16344–16359, 2022.

Dong, J., Feng, B., Guessous, D., Liang, Y., and He, H. Flex attention: A programming model for generating optimized attention kernels. *CoRR*, abs/2412.05496, 2024. doi: 10.48550/ARXIV.2412.05496. URL https://doi.org/10.48550/arXiv.2412.05496.

Fedus, W., Zoph, B., and Shazeer, N. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *J. Mach. Learn. Res.*, 23:120:1–120:39, 2022. URL https://jmlr.org/papers/v23/21-0998.html.

Frantar, E., Ashkboos, S., Hoefler, T., and Alistarh, D. Gptq: Accurate post-training quantization for generative pretrained transformers. *arXiv preprint arXiv:2210.17323*, 2022.

Guo, T., Chen, X., Wang, Y., Chang, R., Pei, S., Chawla, N. V., Wiest, O., and Zhang, X. Large language model based multi-agents: A survey of progress and challenges. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence, IJCAI 2024, Jeju, South Korea, August 3-9, 2024*, pp. 8048–8057. ijcai.org, 2024. URL https://www.ijcai.org/proceedings/2024/890.

He, H. and Lab, T. M. Defeating nondeterminism in llm inference. *Thinking Machines Lab: Connectionism*, 2025. doi: 10.64434/tml.20250910. https://thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference/.

Jia, M. Aime2024, 2024. URL https://huggingface.co/datasets/Maxwell-Jia/AIME_2024.

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with pagedattention. In Flinn, J., Seltzer, M. I., Druschel, P., Kaufmann, A., and Mace, J. (eds.), *Proceedings of the 29th Symposium on Operating Systems Principles, SOSP 2023, Koblenz, Germany, October 23-26, 2023*, pp. 611–626. ACM, 2023. doi: 10.1145/3600006.3613165. URL https://doi.org/10.1145/3600006.3613165.

Li, H., Dong, Q., Chen, J., Su, H., Zhou, Y., Ai, Q., Ye, Z., and Liu, Y. Llms-as-judges: A comprehensive survey on llm-based evaluation methods. *CoRR*, abs/2412.05579, 2024. doi: 10.48550/ARXIV.2412.05579. URL https://doi.org/10.48550/arXiv.2412.05579.

Lin, J., Tang, J., Tang, H., Yang, S., Chen, W.-M., Wang, W.-C., Xiao, G., Dang, X., Gan, C., and Han, S. Awq: Activation-aware weight quantization for on-device llm compression and acceleration. *Proceedings of machine learning and systems*, 6:87–100, 2024.

Liu, Z., Yuan, J., Jin, H., Zhong, S., Xu, Z., Braverman, V., Chen, B., and Hu, X. Kivi: A tuning-free asymmetric 2bit quantization for kv cache. *arXiv preprint arXiv:2402.02750*, 2024.

LMSYS Organization. Sglang: Deterministic inference in large language model serving. https://lmsys.org/blog/2025-09-22-sglang-deterministic/, September 2025. Accessed: 2025-10-29.

Meta AI. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024. URL https://arxiv.org/abs/2407.21783.

Mistral AI. Mistral 7b instruct v0.3: Open-weight instruction-tuned model. https://mistral.ai/news/mistral-7b/, 2024. Accessed: 2025-10-30.

NVIDIA Corporation. Efficient gemm in CUTLASS. NVIDIA Developer Documentation. URL https://docs.nvidia.com/cutlass/media/docs/cpp/efficient_gemm.html.

NVIDIA Corporation. Floating Point and IEEE 754, 2025. URL https://docs.nvidia.com/cuda/floating-point/index.html.

Sheng, G., Zhang, C., Ye, Z., Wu, X., Zhang, W., Zhang, R., Peng, Y., Lin, H., and Wu, C. Hybridflow: A flexible and efficient rlhf framework. In *Proceedings of the Twentieth European Conference on Computer Systems*, pp. 1279–1297, 2025.

Team, Q. Qwen3 technical report, 2025. URL https://arxiv.org/abs/2505.09388.

Thakur, A. S., Choudhary, K., Ramayapally, V. S., Vaidyanathan, S., and Hupkes, D. Judging the judges: Evaluating alignment and vulnerabilities in LLMs-as-judges. In Arviv, O., Clinciu, M., Dhole, K., Dror, R., Gehrmann, S., Habba, E., Itzhak, I., Mille, S., Perlitz, Y., Santus, E., Sedoc, J., Shmueli Scheuer, M., Stanovsky, G., and Tafjord, O. (eds.), *Proceedings of the Fourth Workshop on Generation, Evaluation and Metrics (GEM²)*, pp. 404–430, Vienna, Austria and virtual meeting, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-261-9. URL https://aclanthology.org/2025.gem-1.33/.

Tillet, P., Kung, H., and Cox, D. D. Triton: an intermediate language and compiler for tiled neural network computations. In Mattson, T., Muzahid, A., and Solar-Lezama, A. (eds.), *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages, MAPL@PLDI 2019, Phoenix, AZ, USA, June 22, 2019*, pp. 10–19. ACM, 2019. doi: 10.1145/3315508.3329973. URL https://doi.org/10.1145/3315508.3329973.

tse Huang, J., Zhou, J., Jin, T., Zhou, X., Chen, Z., Wang, W., Yuan, Y., Lyu, M. R., and Sap, M. On the resilience of llm-based multi-agent collaboration with faulty agents, 2025. URL https://arxiv.org/abs/2408.00989.

vLLM Project. Issue #18547: Discussion on deterministic inference and chunked prefill in vllm. https://github.com/vllm-project/vllm/issues/18547, 2025. Accessed: 2025-10-30.

Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., Zheng, C., Liu, D., Zhou, F., Huang, F., Hu, F., Ge, H., Wei, H., Lin, H., Tang, J., Yang, J., Tu, J., Zhang, J., Yang, J., Yang, J., Zhou, J., Lin, J., Dang, K., Bao, K., Yang, K., Yu, L., Deng, L., Li, M., Xue, M., Li, M., Zhang, P., Wang, P., Zhu, Q., Men, R., Gao, R., Liu, S., Luo, S., Li, T., Tang, T., Yin, W., Ren, X., Wang, X., Zhang, X., Ren, X., Fan, Y., Su, Y., Zhang, Y., Zhang, Y., Wan, Y., Liu, Y., Wang, Z., Cui, Z., Zhang, Z., Zhou, Z., and Qiu, Z. Qwen3 technical report. *CoRR*, abs/2505.09388, 2025. doi: 10.48550/ARXIV.2505.09388. URL https://doi.org/10.48550/arXiv.2505.09388.

Yao, F., Liu, L., Zhang, D., Dong, C., Shang, J., and Gao, J. Your efficient rl framework secretly brings you off-policy rl training, August 2025a. URL https://fengyao.notion.site/off-policy-rl.

Yao, F., Liu, L., Zhang, D., Dong, C., Shang, J., and Gao, J. Your efficient rl framework secretly brings you off-policy rl training, August 2025b. URL https://fengyao.notion.site/off-policy-rl.

Yu, G.-I., Jeong, J. S., Kim, G.-W., Kim, S., and Chun, B.-G. Orca: A distributed serving system for {Transformer-Based} generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pp. 521–538, 2022.

Yu, H., Ren, M., Maher, B., Nay, S., Zhu, G., and Jiang, S. Enabling advanced gpu features in pytorch – warp specialization. https://pytorch.org/blog/warp-specialization/, February 2025. PyTorch Blog.

Yuan, J., Liu, H., Zhong, S., Chuang, Y.-N., Li, S., Wang, G., Le, D., Jin, H., Chaudhary, V., Xu, Z., et al. Kv cache compression, but what must we give in return? a comprehensive benchmark of long context capable approaches. *arXiv preprint arXiv:2407.01527*, 2024.

Yuan, J., Li, H., Ding, X., Xie, W., Li, Y., Zhao, W., Wan, K., Shi, J., Hu, X., and Liu, Z. Give me FP32 or give me death? challenges and solutions for reproducible reasoning. *CoRR*, abs/2506.09501, 2025a. doi: 10.48550/ARXIV.2506.09501. URL https://doi.org/10.48550/arXiv.2506.09501.

Yuan, J., Zhang, J., Wen, A., and Hu, X. The science of evaluating foundation models. *arXiv preprint arXiv:2502.09670*, 2025b.

Zhao, Y., Gu, A., Varma, R., Luo, L., Huang, C., Xu, M., Wright, L., Shojanazeri, H., Ott, M., Shleifer, S., Desmaison, A., Balioglu, C., Damania, P., Nguyen, B., Chauhan, G., Hao, Y., Mathews, A., and Li, S. Pytorch FSDP: experiences on scaling fully sharded data parallel. *Proc. VLDB Endow.*, 16(12):3848–3860, 2023. doi: 10.14778/3611540.3611569. URL https://www.vldb.org/pvldb/vol16/p3848-huang.pdf.

Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E., et al. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in neural information processing systems*, 36: 46595–46623, 2023.

Zheng, L., Yin, L., Xie, Z., Sun, C., Huang, J., Yu, C. H., Cao, S., Kozyrakis, C., Stoica, I., Gonzalez, J. E., Barrett, C. W., and Sheng, Y. Sglang: Efficient execution of structured language model programs. In Globersons, A., Mackey, L., Belgrave, D., Fan, A., Paquet, U., Tomczak, J. M., and Zhang, C. (eds.), *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, 2024. URL http://papers.nips.cc/paper_files/paper/2024/hash/724be4472168f31ba1c9ac630f15dec8-Abstract-Conference.html.

A ALGORITHM

A.1 Pseudocode for Tensor-Parallel Invariant Matmul Kernel

Algorithm 1 Tensor-Parallel Invariant MatMul Kernel

Require: Matrices $A \in \mathbb{R}^{M\times K}$, $B \in \mathbb{R}^{K\times N}$, block sizes B_M, B_N, B_K, total tiles $T = K/B_K$, first level tile count K_{first}, reduction depth $L = \log_2(T/K_{first}) + 1$
Ensure: Output matrix $C \in \mathbb{R}^{M\times N}$
for all blocks (m, n) in grid $(\lceil M/B_M \rceil, \lceil N/B_N \rceil)$ **in parallel do**
  Initialize local accumulator: acc$[B_M][B_N] \leftarrow 0$
  Initialize scratch buffers: S$[L][B_M][B_N] \leftarrow 0$
  Initialize counters: Count$[L] \leftarrow 0$
  for $t = 0$ to $T - 1$ **do**
    Load tile $A_t = A[m{:}m + B_M, tB_K : (t + 1)B_K]$
    Load tile $B_t = B[tB_K : (t + 1)B_K, n : n + B_N]$
    acc $\leftarrow$ acc $+ A_tB_t$
    $level \leftarrow 0$
    while $level < L$ **do**
      if ($level = 0$ **and** Count$[level] + 1 = K_{\text{first}}$) **or** ($level > 0$ **and** Count$[level] + 1 = 2$) **then**
        acc $\leftarrow$ acc $+ S[level]$
        $S[level] \leftarrow 0$
        Count$[level] \leftarrow 0$
        $level \leftarrow level + 1$
      else
        $S[level] \leftarrow$ acc
        Count$[level] \leftarrow$ Count$[level] + 1$
        break
      end if
    end while
  end for
  $C[m : m + B_M, n : n + B_N] \leftarrow$ acc
end for

A.2 Pseudocode for Tree All-Reduce

We do not directly adopt NCCL's built-in tree all-reduce algorithm because NCCL only allows users to specify a tree topology **across nodes**, while **within each node** it still defaults to a chain-based reduction. In typical deployment scenarios, tensor parallelism is applied to multiple GPUs within the same node. Therefore, to ensure that reductions within a node also strictly follow a tree-structured topology, we implement a customized tree all-reduce algorithm as shown in 2.

Algorithm 2 Customized Tree All-Reduce

Require: Local results x, world size W
Ensure: Tree All-Reduced Results
  Allocate list $R[0 : W - 1]$
  Initialize each $R[i] \leftarrow 0$ with shape of x
  AllGather: collect all x into R
  for $\ell = 1$ to bit_length$(W) - 1$ **do**
    for $left = 0$ to $W - 1$ step 2^ℓ **do**
      $right \leftarrow left + 2^{\ell-1}$
      $R[left] \leftarrow R[left] + R[right]$
    end for
  end for
  return $R[0]$

B SUPPLEMENTARY EXPERIMENT SETUP DETAILS

B.1 Decoding Parameters for Random Sampling

The decoding parameters are set to (temperature = 0.6, top-p = 0.95, top-k = 20) for reasoning models, and (temperature = 0.7, top-p = 0.8, top-k = 20) for non-reasoning models following the official best practice provided in the Qwen3 Techinical Report (Team, 2025).

B.2 Deterministic Inference in vLLM

To enable deterministic inference with `vLLM`, all our experiments are conducted in eager mode, under which CUDA Graphs are disabled, and prefix caching is turned off. We fix the random seed to 42.

However, the `vLLM` v1 engine enables chunked prefill by default and does not allow users to disable it (vLLM Project, 2025). This chunking strategy potentially conflicts with the requirements for deterministic inference (LMSYS Organization, 2025). To efficiently handle long prompts under limited GPU memory, vLLM divides the prefill process of a sequence into multiple chunks. This chunked execution alters the computation and scheduling order—even when the underlying computation kernels are deterministic. Moreover, this process is invisible to users and cannot be explicitly controlled.

We observed that these implicit optimizations can indeed affect model outputs in practice. When the model size or batch size are large, GPU memory is constrained. Under these conditions, the inference process is not run-to-run deterministic, and batch-invariant operations (BIO) cannot fully ensure reproducible outputs when the batch size changes, even at TP=1. To eliminate the impact of such serving-system scheduling behaviors on our method evaluation, we set `max_num_seqs=1` for the Qwen3-32B experiments on NVIDIA L40S with TP=2.

C Supplementary Reproducibility Evaluation Results

C.1 Average count of unique answer on NVIDIA RTX PRO 6000 GPU

Table 4. **Average Count of Unique Outputs on AIME24 and AMC23.** For each prompt, outputs are generated under 12 runtime configurations (BS=8/16/32; TP=1/2/4/8). Qwen3-32B was evaluated under 9 configurations (BS=8/16/32; TP=2/4/8). "1" indicates identical outputs across all configurations.

Model	Method	AIME'24	AMC'23
Qwen3-8B	BF16	12.00	11.20
	BIO	7.87	7.53
	BIO+TBIK	**1**	**1**
Mistral-7B-Instruct	BF16	11.97	11.13
	BIO	7.97	7.58
	BIO+TBIK	**1**	**1**
Llama-3.1-8B-Instruct	BF16	9.37	9.55
	BIO	7.00	6.80
	BIO+TBIK	**1**	**1**
Qwen3-32B	BF16	9.00	8.20
	BIO	6.87	6.73
	BIO+TBIK	**1**	**1**

As shown in Tables 4, vanilla BF16 inference exhibits high non-determinism under varying system configurations. The BIO can partially mitigate this issue by reducing the average number of unique outputs, as consistent results are achieved when the TP size is 1 or 2 with varying batch sizes. When combining BIO with TBIK, the average number of unique outputs across all models on both the AIME24 and AMC23 datasets drops to one, indicating fully deterministic outputs under all tested configurations. This observation is consistent with our experimental findings on NVIDIA L40S GPUs.

C.2 Average maximum probability divergence on NVIDIA RTX PRO 6000 GPU

Table 5 present the average Maximum Probability Divergence on two datasets. Compared with vanilla BF16 inference, BIO slightly reduces the fluctuation of token predictive probabilities due to its parallel strategy and control over reduction operations. However, it still fails to handle variations caused by changes in TP. In contrast, BIO+TBIK achieves a strictly zero Maximum Probability Divergence across all experimental settings, demonstrating bit-wise deterministic LLM inference. This observation is consistent with our experimental findings on NVIDIA L40S GPUs.

Table 5. **Average Maximum Probability Divergence on AIME24 and AMC23($\times 10^{-3}$).** A larger value indicates greater numerical discrepancies, while 0 means the predicted token probability distributions are bitwise identical across all evaluated runtime configurations.

Model	Method	AIME'24	AMC'23
Qwen3-8B	BF16	9.72	7.63
	BIO	8.35	7.14
	BIO+TBIK	**0**	**0**
Mistral-7B-Instruct	BF16	15.58	19.48
	BIO	14.85	17.44
	BIO+TBIK	**0**	**0**
Llama-3.1-8B-Instruct	BF16	31.84	27.04
	BIO	31.65	25.95
	BIO+TBIK	**0**	**0**
Qwen3-32B	BF16	9.84	9.34
	BIO	8.32	8.57
	BIO+TBIK	**0**	**0**

D Supplementary Performance Evaluation Results

D.1 Overhead of Tree-Based All-Reduce

From Figure 11, we can observe that our tree-based all-reduce implementation introduces noticeable overhead compared to NCCL all-reduce, ranging from roughly 60% to 180%. The impact is especially significant during the prefill stage. This high overhead is largely due to our naive implementation and can be reduced by implementing a custom tree-structured all-reduce in NCCL.

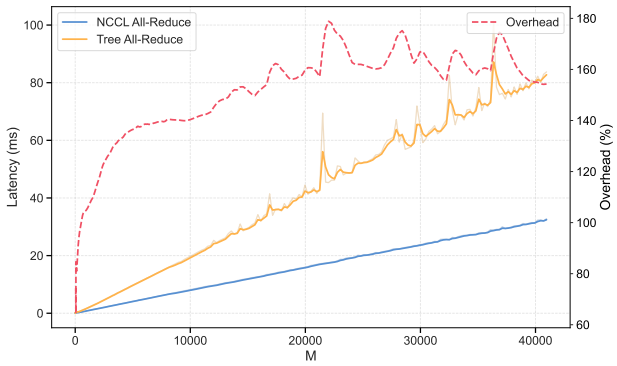


Figure 11. Latency comparison of our Tree-Based all-reduce and NCCL all-reduce for varying M, with $N = 4096$ on four NVIDIA L40S GPUs without NVLink.

D.2 Fine-grained Latency Breakdown of TBIK

To better understand the contributions of the two core components in TBIK—the tree-based MatMul kernel and the

tree-based all-reduce operation—to the overall end-to-end performance, we evaluate four configurations: (1) vanilla BF16 inference, (2) tree-based MatMul only (without tree all-reduce), (3) tree-based all-reduce only (without tree matmul), and (4) both components enabled.

Figure 12 presents the latency breakdown across different input/output lengths. Although the tree-based MatMul exhibits lower throughput compared with the cuBLAS BF16 implementation, it introduces only 3–14% overhead. This limited impact arises because the row-split linear layer accounts for only a small fraction of the model's total computation, and therefore its inefficiency has minimal effect on end-to-end latency.

In contrast, the tree-based all-reduce operation contributes a substantially larger overhead of 28–50%. This cost is primarily attributed to our current unoptimized custom implementation, in combination with the absence of NVLink in the evaluation environment, which significantly degrades communication efficiency across GPUs.

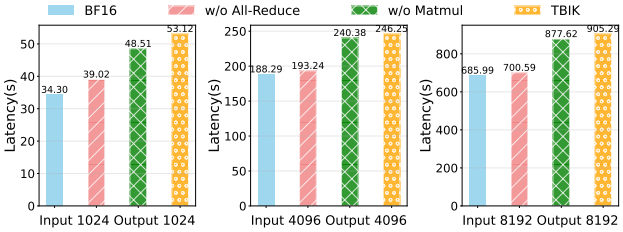


Figure 12. Fine-grained end-to-end latency breakdown of the Qwen3-8B model on four NVIDIA L40s GPUs under varying input/output lengths with a batch size of 64.

E TBIK BLOCK SIZE SELECTION

In this section, we present the block sizes used in TBIK, namely $Block_M$, $Block_N$, and $Block_K$. To achieve optimal throughput, we employ different configurations for different data types, as summarized in Table 6. The block sizes are determined via a grid search over the values $[16, 32, 64, 128, 256]$ on NVIDIA L40S GPUs, and for each data type we select the configuration that yields the highest throughput.

Table 6. Block size setting for TBIK.

Dtype	$BLOCK_M$	$BLOCK_K$	$BLOCK_N$
BF16	64	256	128
FP16	64	256	128
FP32	32	128	64